

Atividade Prática 6 – Detector de Máximo e Mínimo

Pedro Lucas dos Reis Silva - RA: 2272040

Lucas dos Reis Viana - RA: 2321645

6 de junho de 2026

1 Introdução

Esta atividade consiste na implementação, em VHDL, de um detector de valor mínimo e valor máximo para um conjunto de valores inteiros. O circuito recebe quatro entradas sem sinal de 2 bits e produz duas saídas: o menor valor encontrado e o maior valor encontrado.

Os conceitos novos utilizados foram:

- **package**: arquivo separado usado para declarar tipos e subprogramas compartilhados pelo projeto;
- **type**: criação de um tipo de vetor de inteiros, utilizado para armazenar internamente os valores de entrada;
- **procedure**: subprograma responsável por percorrer o vetor e calcular mínimo e máximo;
- **generic**: parâmetros usados para definir a quantidade de entradas e a largura de bits dos valores.

2 Desenvolvimento

2.1 Arquivos do Projeto

O projeto foi dividido em dois arquivos VHDL:

- **meupacote.vhd**: contém os tipos auxiliares e o procedimento `detector_min_max`;
- **atividade6.vhd**: contém a entidade principal, as portas de entrada e saída e a chamada do procedimento.

A entidade principal é **atividade6**. Ela possui os genéricos `NUM_INPUTS` e `NUM_BITS`, com valores padrão iguais a 4 e 2, respectivamente. Assim, a versão implementada compara quatro valores de 2 bits.

2.2 Entradas e Saídas

As entradas e saídas externas foram declaradas como `std_logic_vector`, garantindo compatibilidade com o arquivo de formas de onda utilizado na simulação e com as atribuições de pinos do Quartus Prime. As portas são:

- **e1, e2, e3, e4**: entradas de 2 bits;

- `saida_min`: menor valor entre as quatro entradas;
- `saida_max`: maior valor entre as quatro entradas.

Dentro da arquitetura, as entradas são convertidas para inteiros e armazenadas no sinal `x`, do tipo `int_array`. Esse tipo foi declarado no pacote `meupacote`.

2.3 Funcionamento Lógico

O procedimento `detector_min_max` inicia o valor mínimo e o valor máximo com o primeiro elemento do vetor de entrada. Em seguida, um laço `for` percorre todos os elementos do vetor. Se o elemento atual for menor que o mínimo armazenado, o mínimo é atualizado. Se for maior que o máximo armazenado, o máximo é atualizado.

Na entidade principal, o procedimento é chamado dentro de um `process(x)`. Dessa forma, sempre que alguma entrada muda, o vetor interno `x` é atualizado e o procedimento recalcula as saídas.

Ao final, os valores calculados como inteiros são convertidos para `unsigned` e, em seguida, para `std_logic_vector`, formando as saídas `saida_min` e `saida_max`.

2.4 Implementação na Placa DE10-Lite

Na implementação física, as quatro entradas de 2 bits utilizam as chaves SW0 a SW7. As saídas são apresentadas nos LEDs vermelhos: LEDR0 e LEDR1 indicam o menor valor, enquanto LEDR8 e LEDR9 indicam o maior valor. A Tabela 1 apresenta as atribuições definidas no arquivo `atividade6.qsf`. Todos os sinais utilizam o padrão elétrico 3.3-V LVTTTL.

Tabela 1: Pinagem do detector de mínimo e máximo na placa DE10-Lite.

Recurso	Sinal VHDL	Função	Pino
SW0	<code>e1(0)</code>	bit 0 da entrada 1	PIN_C10
SW1	<code>e1(1)</code>	bit 1 da entrada 1	PIN_C11
SW2	<code>e2(0)</code>	bit 0 da entrada 2	PIN_D12
SW3	<code>e2(1)</code>	bit 1 da entrada 2	PIN_C12
SW4	<code>e3(0)</code>	bit 0 da entrada 3	PIN_A12
SW5	<code>e3(1)</code>	bit 1 da entrada 3	PIN_B12
SW6	<code>e4(0)</code>	bit 0 da entrada 4	PIN_A13
SW7	<code>e4(1)</code>	bit 1 da entrada 4	PIN_A14
LEDR0	<code>saida_min(0)</code>	bit 0 do mínimo	PIN_A8
LEDR1	<code>saida_min(1)</code>	bit 1 do mínimo	PIN_A9
LEDR8	<code>saida_max(0)</code>	bit 0 do máximo	PIN_A11
LEDR9	<code>saida_max(1)</code>	bit 1 do máximo	PIN_B11

3 Simulação

A simulação apresenta as quatro entradas do circuito, `e1`, `e2`, `e3` e `e4`, juntamente com as saídas `saida_min` e `saida_max`. Os estímulos aplicados variam os valores de entrada para verificar situações com valores crescentes, decrescentes, repetidos e iguais. Em todos os casos, a saída

`saida_min` corresponde ao menor valor presente no conjunto de entradas, enquanto `saida_max` corresponde ao maior valor.

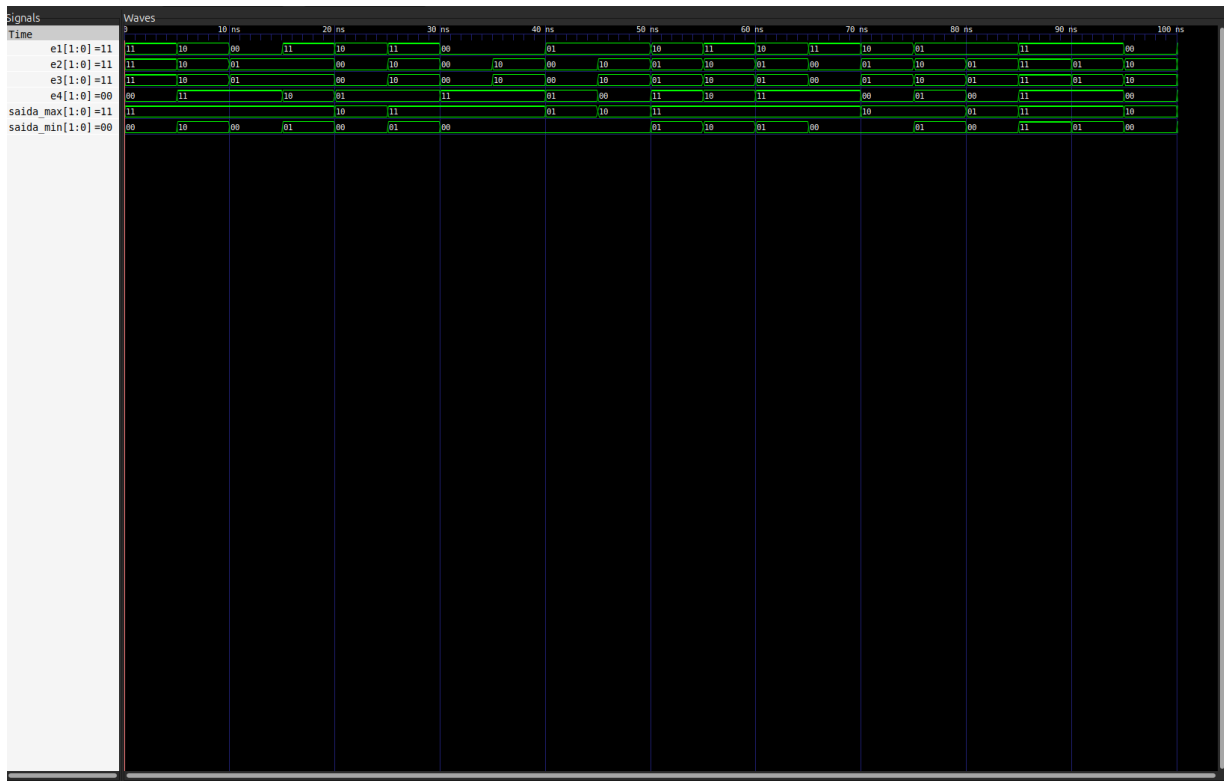


Figura 1: Formas de onda da simulação do detector de valor mínimo e máximo.

4 Diagrama RTL

O diagrama RTL gerado pelo Quartus Prime representa a entidade `atividade6` e a lógica combinacional responsável pela comparação dos valores de entrada. A estrutura sintetizada recebe os quatro valores convertidos para o vetor interno e produz os sinais de menor e maior valor, correspondentes às saídas `saida_min` e `saida_max`.

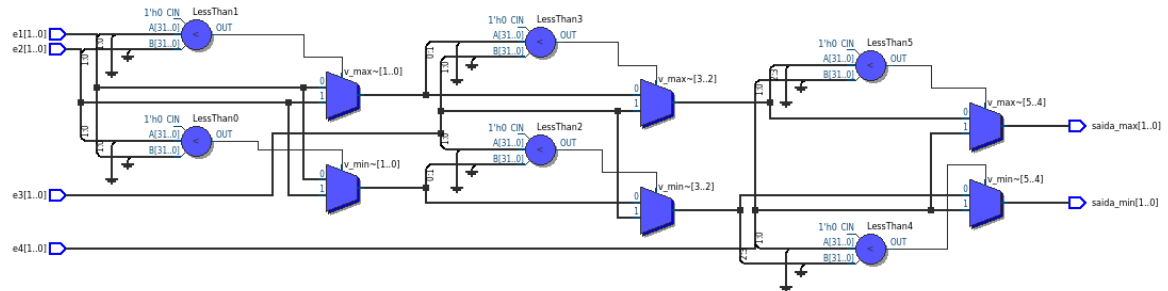


Figura 2: Diagrama RTL do detector de valor mínimo e máximo.

5 Considerações Finais

A atividade permitiu implementar um detector de mínimo e máximo utilizando `package`, `type`, `procedure` e `generic`. O pacote separa as declarações auxiliares da entidade principal, enquanto o procedimento concentra a lógica de comparação. A solução recebe quatro valores de 2 bits, converte-os para um vetor interno de inteiros, calcula os extremos e retorna os resultados nas saídas. A simulação confirmou o comportamento esperado e a atribuição de pinos possibilitou a implementação do circuito na placa DE10-Lite, utilizando chaves para as entradas e LEDs para a indicação dos valores mínimo e máximo.

A Código VHDL – Pacote meupacote

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 package meupacote is
6     -- Definição de tipo array de std_logic_vector conforme pedido
7     type slv_array is array (natural range <>) of unsigned;
8
9     -- Definição de tipo array de integer conforme pedido
10    type int_array is array (natural range <>) of integer;
11
12    -- Procedimento para encontrar min e max em um array de inteiros
13    procedure detector_min_max (
14        signal entrada : in int_array;
15        signal min_val : out integer;
16        signal max_val : out integer
17    );
18 end package meupacote;
19
20 package body meupacote is
21     procedure detector_min_max (
22         signal entrada : in int_array;
23         signal min_val : out integer;
24         signal max_val : out integer
25     ) is
26         variable v_min : integer;
27         variable v_max : integer;
28     begin
29         -- Inicializa com o primeiro elemento
30         v_min := entrada(entrada'low);
31         v_max := entrada(entrada'low);
32
33         -- Loop para percorrer o array e comparar valores
34         for i in entrada'range loop
35             if entrada(i) < v_min then
36                 v_min := entrada(i);
37             end if;
38             if entrada(i) > v_max then
39                 v_max := entrada(i);
40             end if;
41         end loop;
42
43         -- Atribui os resultados das variáveis aos sinais de saída
44         min_val <= v_min;
45         max_val <= v_max;
46     end procedure;
47 end package body meupacote;

```

Listing 1: Arquivo meupacote.vhd.

B Código VHDL – Entidade atividade6

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4 use work.meupacote.all; -- Importando o pacote
5
6 entity atividade6 is
7     generic (
8         NUM_INPUTS : integer := 4; -- Quantidade genérica de valores
9         NUM_BITS    : integer := 2 -- Quantidade genérica de bits (2 p/
10             caber nos 10 SW da DE10-Lite)
11     );
12     port (
13         -- Interface em std_logic_vector para compatibilidade com o VWF
14         e1, e2, e3, e4 : in  std_logic_vector(NUM_BITS-1 downto 0);
15         -- Saídas de valor mínimo e máximo
16         saida_min      : out std_logic_vector(NUM_BITS-1 downto 0);
17         saida_max      : out std_logic_vector(NUM_BITS-1 downto 0)
18     );
19 end entity;
20
21 architecture comportamento of atividade6 is
22     -- Sinal interno utilizando o tipo definido no package
23     signal x : int_array(0 to NUM_INPUTS-1);
24     signal m_val, M_val_int : integer;
25 begin
26     -- Mapeamento das portas std_logic_vector para o vetor de inteiros
27     x(0) <= to_integer(unsigned(e1));
28     x(1) <= to_integer(unsigned(e2));
29     x(2) <= to_integer(unsigned(e3));
30     x(3) <= to_integer(unsigned(e4));
31
32     -- O process e uma instrucao concorrente da arquitetura.
33     -- Dentro dele, a chamada do procedure e sequencial e recalcula min
34     /max quando x muda.
35     process(x)
36     begin
37         detector_min_max(x, m_val, M_val_int);
38     end process;
39
40     -- Converte os inteiros para std_logic_vector nas saídas
41     saida_min <= std_logic_vector(to_unsigned(m_val, NUM_BITS));
42     saida_max <= std_logic_vector(to_unsigned(M_val_int, NUM_BITS));
43 end architecture;

```

Listing 2: Arquivo atividade6.vhd.